UPLA

Arquitectura Física de Almacenamiento y Distribución de Archivos

Administración de Instancias, Estructuras de Almacenamiento y Gestión de Datos Masivos – **Semana 06**

CAPACIDAD

Gestionar las instancias del servidor, estructuras de almacenamiento y esquemas de una base de datos para trabajar con datos de cualquiera de las bases administradas por una instancia y manejar grandes cantidades de datos de manera eficiente.

DESEMPEÑO

Diseña e implementa la distribución física de la base de datos del proyecto, fragmentando y asignando estratégicamente archivos primarios (.mdf), secundarios (.ndf) y registros de transacciones (.ldf) en unidades de disco dedicadas para mejorar el rendimiento de lectura/escritura.

OBJETIVO DEL APRENDIZAJE

Diseñar e implementar la distribución física de la base de datos del proyecto corporativo en unidades de disco o particiones dedicadas, para maximizar la concurrencia, balancear la E/S de disco (I/O) y optimizar el rendimiento general de lectura y escritura en el DBMS (SQL Server).



UPLA

Fundamento Teórico: Arquitectura Física en SQL Server



En sistemas empresariales, un bajo rendimiento no siempre se debe a consultas mal optimizadas, sino a una mala distribución física del almacenamiento.

COMPONENTES FÍSICOS DE UNA BASE DE DATOS

Archivo Primario (.mdf)

Contiene el punto de inicio de la base de datos y los metadatos del sistema. Almacena también los objetos de usuario del grupo PRIMARY.

Archivos Secundarios (.ndf)

Permiten escalar la base de datos más allá de un solo disco. Se agrupan en Filegroups para repartir tablas o índices grandes y paralelizar el acceso.

Registro de Transacciones (.ldf)

Contiene el registro secuencial de todas las modificaciones (Write-Ahead Logging). Conviene aislarlo en un disco dedicado para evitar contención de cabezales (I/O contention).

ESTRATEGIA DE DISCOS DEDICADOS (SEPARACIÓN DE I/O)

Unidad DATA

Archivos primarios y secundarios de datos masivos.

Unidad LOG

Exclusivamente los registros de transacciones (.ldf). Como las escrituras en el log son secuenciales, un disco separado acelera inserciones, actualizaciones y transacciones masivas.

Unidad INDEX (opcional avanzado)

Índices pesados, separando el acceso a las tablas del de los índices de búsqueda rápida.

Base de datos GradosTitulos



UPLA

Caso Práctico GradosTitulos: Creación de Grupos de Archivos (Filegroups) y Base de Datos Multidisco

Por defecto toda base de datos usa el grupo PRIMARY.
Se crean filegroups dedicados: Académico, Trámites y Seguridad/Auditoría.
El log de transacciones se aísla en su propio archivo .ldf.

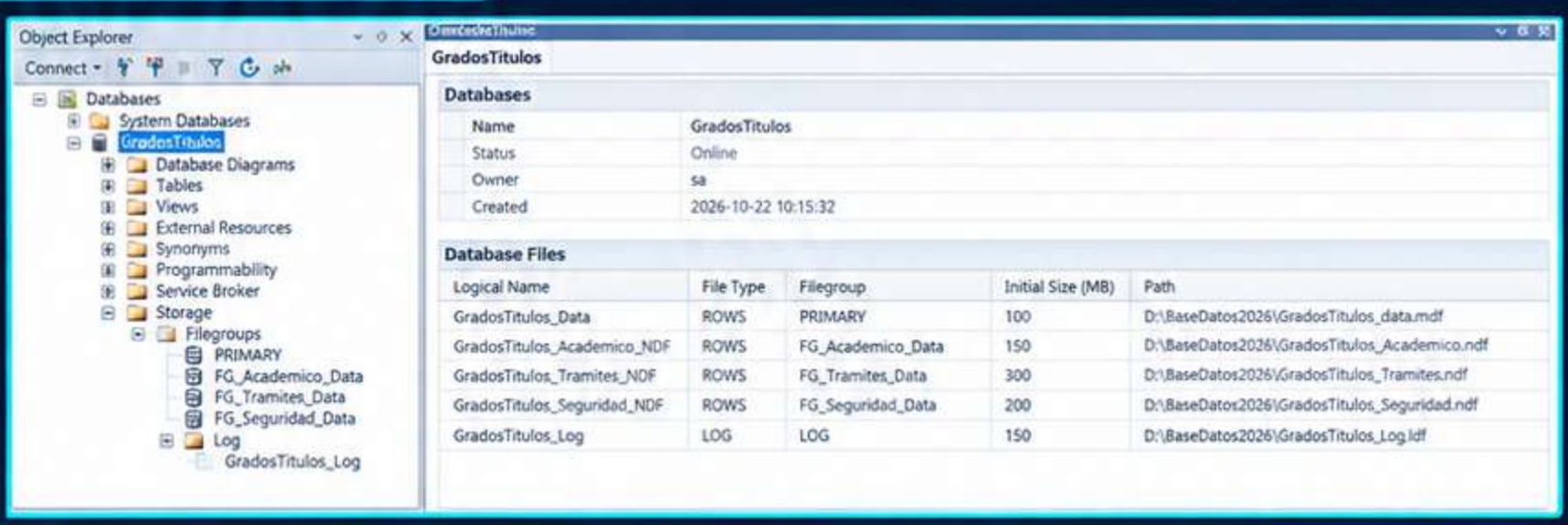
Grupos de archivos y tamaños



SCRIPT

```
USE master;
GO
CREATE DATABASE GradosTitulos
ON PRIMARY
(
      NAME = GradosTitulos_Data,
      FILENAME = 'D:\BaseDatos2026\GradosTitulos_data.mdf',
      SIZE = 100MB,
      MAXSIZE = UNLIMITED,
      FILEGROWTH = 50MB
),
FILEGROUP FG_Academico_Data
(
      NAME = 'GradosTitulos_Academico_NDF',
      FILENAME = 'D:\BaseDatos2026\GradosTitulos_Academico.ndf',
      SIZE = 150MB,
      MAXSIZE = UNLIMITED,
      FILEGROWTH = 50MB
),
FILEGROUP FG_Tramites_Data
(
      NAME = 'GradosTitulos_Tramites_NDF',
      FILENAME = 'D:\BaseDatos2026\GradosTitulos_Tramites.ndf',
      SIZE = 300MB,
      MAXSIZE = UNLIMITED,
      FILEGROWTH = 100MB
),
FILEGROUP FG_Seguridad_Data
(
      NAME = 'GradosTitulos_Seguridad_NDF',
      FILENAME = 'D:\BaseDatos2026\GradosTitulos_Seguridad.ndf',
      SIZE = 200MB,
      MAXSIZE = UNLIMITED,
      FILEGROWTH = 64MB
).
LOG ON
(
      NAME = 'GradosTitulos_Log',
      FILENAME = 'D:\BaseDatos2026\GradosTitulos_Log.ldf',
      SIZE = 150MB,
      MAXSIZE = 2GB,
      FILEGROWTH = 50MB
);
GO
```

Resultado en SQL Server



UPLA

Asignación Estratégica de Tablas a los Grupos de Archivos

Para que la distribución física mejore el rendimiento, cada tabla clave se aloja explícitamente en su filegroup al definir su PRIMARY KEY. Persona va en FG_Academico_Data y la auditoría, aislada en FG_Seguridad_Data para no degradar las tablas transaccionales.



</> SCRIPT

```sql
USE GradosTitulos;
GO
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Academico')
      EXEC('CREATE SCHEMA Academico;');
GO
CREATE TABLE Academico.Persona (
    IdPersona         INT           NOT NULL IDENTITY(1,1),
    DNI               CHAR(8)       NOT NULL,
    Nombres           VARCHAR(80)   NOT NULL,
    ApellidoPaterno   VARCHAR(60)   NOT NULL,
    ApellidoMaterno   VARCHAR(60)   NULL,
    Correo            VARCHAR(120)  NULL,
    Telefono          VARCHAR(15)   NULL,
    TipoPersona       VARCHAR(15)   NOT NULL,  -- ESTUDIANTE, DOCENTE
    FechaRegistto     DATETIM2      NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT PK_Persona PRIMARY KEY CLUSTERED (IdPersona)
        ON FG_Academico_Data,
    CONSTRAINT UQ_Persona_DNI UNIQUE (DNI)
);
GO
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Seguridad')
      EXEC('CREATE SCHEMA Seguridad;');
GO
CREATE TABLE Seguridad.AuditoriaSeguridad (
    IdAuditoria     BIGINT  IDENTITY(1,1) NOT NULL,
    FechaHora       DATETIME2   NOT NULL DEFAULT SYSDATETIME(),
    IdUsuario       INT  NULL,
    NombreUsuario   VARCHAR(60)    NULL,
    TipoEvento      VARCHAR(35)    NOT NULL,
    Resultado       VARCHAR(15)    NOT NULL,
    Modulo          VARCHAR(50)    NULL,
    Accion          VARCHAR(80)    NULL,
    DireccionIP     VARCHAR(45)    NULL,
    Detalle         NVARCHAR(2000) NULL,
    CONSTRAINT PK_AuditoriaSeguridad PRIMARY KEY CLUSTERED (IdAuditoria)
        ON FG_Seguridad_Data
);   )
GO
```

Resultado en SQL Server

